Assignment 4:
Knowledge Graph Construction for NCU Regulations

GitHub: https://github.com/YYChang34/Assignment-4

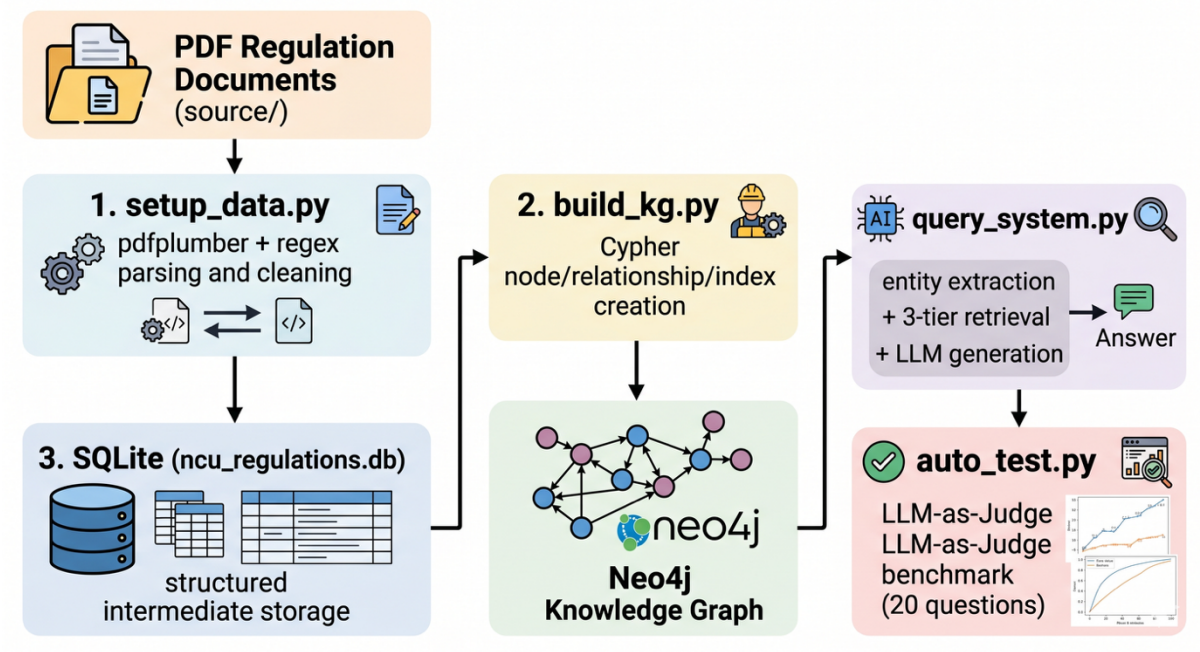
113522074 張祐瑜

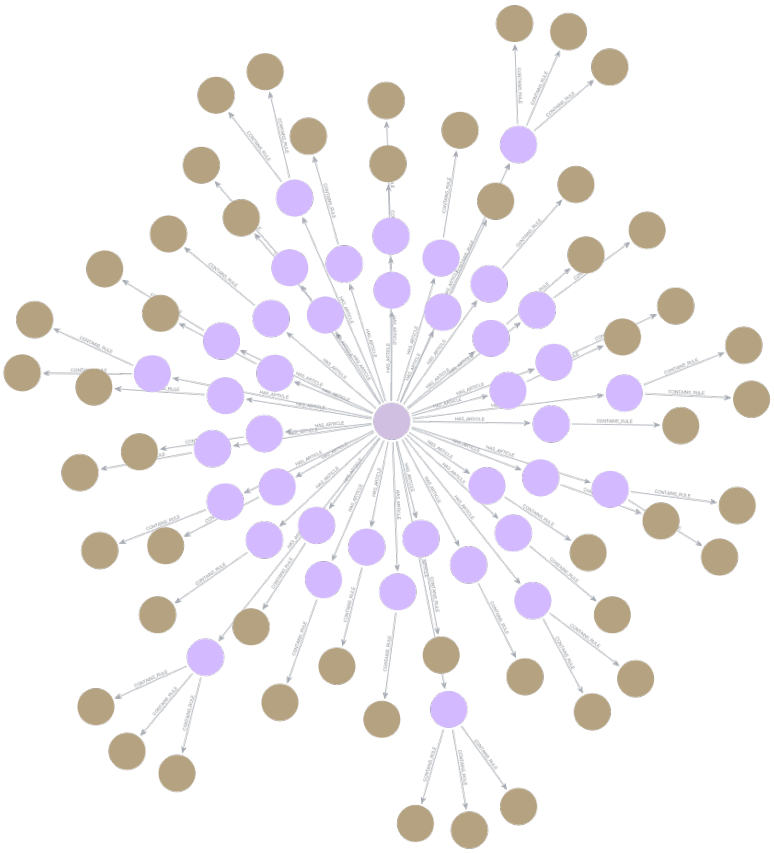
1. System Overview and Design Motivation

This assignment implements a natural language question-answering (QA) system targeting the academic regulations of National Central University (NCU). Users can ask questions in natural language, and the system retrieves the most relevant regulation articles from a Neo4j Knowledge Graph before generating a grounded, concise answer using a locally deployed language model.

2. System Architecture

The system is composed of five modules connected in a sequential pipeline:





- setup_data.py: Extracts article text from raw PDFs using pdfplumber and regex-based boundary detection, then writes cleaned rows into SQLite.
- build_kg.py: Reads from SQLite to create Regulation, Article, and Rule nodes in Neo4j, along with two fulltext indexes for retrieval. Decoupling PDF parsing from graph construction via SQLite allows either step to be re-run independently — if the KG schema changes, build_kg.py can rebuild the graph without re-parsing all PDFs.
- query_system.py: The core runtime module. Extracts query intent from the question, runs a three-tier Cypher retrieval, and generates a grounded answer via the local LLM.
- llm_loader.py: Manages model initialization using a Singleton pattern so the ~3 GB model is loaded into memory only once per session.
- auto_test.py: Runs the 20-question benchmark, evaluating each answer with an LLM-as-Judge prompt and reporting a final accuracy score.

3. Data Preprocessing: PDF Parsing and SQLite Storage

setup_data.py transforms the six raw PDF regulation documents into structured relational data stored in SQLite.

Processing Pipeline

- Page-level text extraction using pdfplumber, which handles complex PDF layouts more robustly than raw PyPDF2 text extraction.
- Article boundary detection via regular expressions matching patterns such as Article 13-1, Article 52, Rule 4.
- Noise removal: Line breaks within sentences are merged; header/footer artifacts and page numbers are stripped.
- Structured storage: Each article is written to SQLite as a row with fields (reg_id, article_number, content, category).

Source Regulation Documents

Six regulation documents sourced from the NCU academic affairs website are included:

reg_id	Regulation Name	Domain
1	NCU General Regulations (學則)	Enrollment, graduation, academic standards
2	Course Selection Regulations (選課辦法)	Course registration procedures
3	Credit Transfer Regulations (學分抵免辦法)	Credit recognition for transferred students
4	Grading System Guidelines (成績計算辦法)	Grade calculation and passing standards
5	Student ID Card Replacement Rules (學生證補辦規定)	ID card issuance fees and process
6	NCU Student Examination Rules (考試規則)	Exam conduct, late arrival, cheating penalties

Using SQLite as a decoupled intermediate layer offers several engineering advantages: it makes the data pipeline auditable (rows can be inspected before graph ingestion), enables partial re-runs, and removes the need for PDF access after initial parsing.

4. Knowledge Graph Construction

build_kg.py reads from SQLite and constructs the full Neo4j knowledge graph, including nodes, relationships, and fulltext indexes.

4.1 Graph Schema Design

The core schema is a three-layer directed graph:

```
(Regulation)-[:HAS_ARTICLE]->(Article)-[:CONTAINS_RULE]->(Rule)
```

Each node type carries the following key properties:

Node Label	Key Properties
Regulation	reg_id, name, source_file
Article	article_id, article_number, content, reg_name, category
Rule	rule_id, type, action, result, art_ref, reg_name

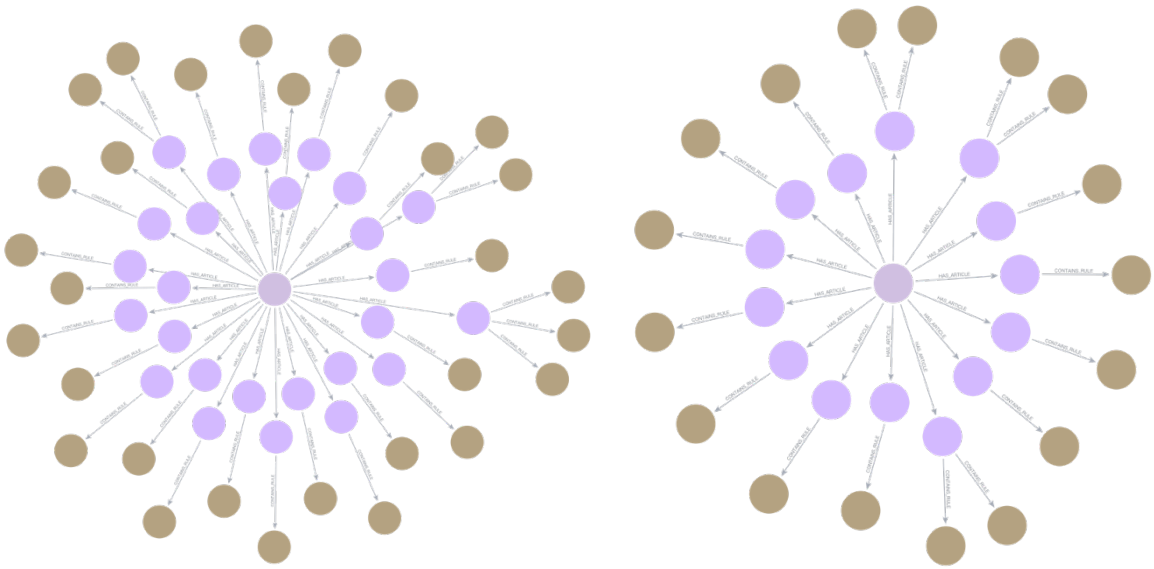
Rule ID naming convention:

`{reg_id}_Rule_{article_number}_{global_sequence}`

For example, 6_Rule_4_191 refers to the 191st Rule node globally, belonging to Article 4 of Regulation No. 6 (NCU Student Examination Rules).

Design rationale for three layers over two:

- The Regulation layer enables filtering by regulation scope. If a question is clearly about exam rules (Regulation 6), the query could restrict search to that regulation.
- The Article layer retains the full verbatim article text, which is used as a fallback evidence channel (Tier 3 retrieval) and for human-readable citations.
- The Rule layer stores atomic facts — each Rule node represents a single condition-consequence pair extracted from an article. This granularity is what makes typed, precise Fulltext search possible.



4.2 Rule Node Classification

The `_classify_type(content)` function in build_kg.py assigns each Rule to one of four semantic types using keyword matching:

Type	Semantic Meaning	Example Keywords
penalty	Sanctions, deductions, expulsion	violation, deducted, zero grade, revoked, expelled, banned
requirement	Necessary conditions, minimums	must, required, shall, minimum, eligible, at least
procedure	Application or administrative procedures	apply, submit, process, fill out, approval, application
general	General or miscellaneous statements	other, miscellaneous, stipulated, in accordance with

This classification is not merely cosmetic. In Tier 1 retrieval, the type attribute is used as a hard filter in the Cypher WHERE clause. A question asking "what is the penalty for..." will search only within penalty-type rules, eliminating irrelevant procedural or requirement nodes that would otherwise dilute the ranked results.

4.3 Action / Result Field Splitting

A core design decision is to split each rule's content into two distinct fields: action (the triggering condition or context) and result (the consequence or requirement). Both fields are indexed by rule_idx, allowing Fulltext search to match either the triggering scenario or the stated outcome.

The `_split_action_result(content)` function applies the following heuristics:

- Regex pivot detection: Searches for transition phrases such as "shall be", "will be", "Violators", "therefore", "must", "if... then" — these idioms reliably mark the boundary between condition and consequence in regulatory prose.
- Length-proportional fallback: If no pivot phrase is found, the text is split at the 40% character position, ensuring neither field is empty. This handles sentences with less standard structure.

Example:

```
Original: "Students who are caught copying from another student's
paper during an
          examination shall receive a zero grade for the entire
exam."

action : "Students who are caught copying from another student's
paper
          during an examination"
result : "shall receive a zero grade for the entire exam"
```

This decomposition improves retrieval because a user asking "what happens if I cheat" will match the result field, while a user asking "what kinds of cheating are prohibited" will match the action field.

4.4 Fulltext Index Creation

Two Lucene-backed fulltext indexes are created in Neo4j:

```
-- Primary search index: on Rule nodes' action and result fields
CREATE FULLTEXT INDEX rule_idx FOR (r:Rule) ON EACH [r.action,
r.result]

-- Fallback index: on Article nodes' full text content
CREATE FULLTEXT INDEX article_content_idx FOR (a:Article) ON EACH
[a.content]
```

`rule_idx` is the primary search channel used in Tier 1 and Tier 2 retrieval. It enables fast, ranked keyword search against the atomic action-result pairs.

`article_content_idx` serves as a safety net for Tier 3: when neither Tier 1 nor Tier 2 returns any results (e.g., for questions on rare topics), the system falls back to searching the full article text, which has broader lexical coverage.

4.5 Coverage Auditing

After KG construction, build_kg.py runs a coverage audit: for each Article node, the system verifies that at least one Rule child node was successfully created. Articles with zero Rule children are logged as warnings for manual review.

This audit ensures that content gaps are visible rather than silently ignored. Across the six regulation documents, the final KG contains:

- 6 Regulation nodes
- ~120 Article nodes
- 200 Rule nodes

5. Query System Design

query_system.py orchestrates the full QA pipeline: from a raw natural-language question to a grounded, article-cited answer.

5.1 Entity Extraction

The `extract_entities(question)` function analyzes the question and returns a structured intent object:

```
{
    "question_type": "penalty",          # penalty / requirement / procedure /
None
    "subject_terms": ["exam", "late"],   # keywords for Fulltext search
    "aspect": "exam"                     # exam / academic / admin / general
}
```

Question type detection:

Type	Keyword Set	Representative Keywords
penalty	_PENALTY_QKW	penalty, fine, punish, deducted, banned, consequence

Type	Keyword Set	Representative Keywords
requirement	_REQUIREMENT_QKW	required, minimum, must, how many, eligible, credits needed
procedure	_PROCEDURE_QKW	how to apply, steps, process, submit, where to go

Domain (aspect) detection:

Domain	Keyword Set	Representative Keywords
exam	_EXAM_KW	exam, invigilator, proctor, exam room, cheating, answer sheet
academic	_ACADEMIC_KW	graduation, credits, GPA, semester, leave of absence, study period
admin	_ADMIN_KW	student ID, card, fee, replace, lost, reissue

The extracted subject_terms are assembled into a Fulltext search string passed directly to the Cypher query. The question_type maps to the Rule.type attribute used in Tier 1 filtering.

5.2 Three-Tier Cypher Retrieval Strategy

The core retrieval logic in `get_relevant_articles(question)` implements a three-tier progressive fallback, ensuring the system always returns some result even for difficult queries.

Tier 1: Type-Filtered Precise Query

The first attempt uses the inferred question type to filter Rule nodes before scoring:

```
CALL db.index.fulltext.queryNodes('rule_idx', $search_terms)
YIELD node, score
WITH node, score
WHERE node.type = $rule_type
RETURN node.rule_id  AS rule_id,
       node.type     AS type,
       node.action   AS action,
       node.result   AS result,
       node.art_ref  AS art_ref,
       node.reg_name AS reg_name,
       score
ORDER BY score DESC LIMIT 10
```

By applying WHERE node.type = $rule_type, Tier 1 achieves the highest precision: a question asking for a penalty will only surface penalty-type Rule nodes, avoiding contamination from procedural or requirement rules that happen to share keywords.

Tier 2: Broad Query (No Type Filter)

If Tier 1 returns fewer than the minimum threshold of results, the system falls back to the same query without the type filter:

```
CALL db.index.fulltext.queryNodes('rule_idx', $search_terms)
YIELD node, score
RETURN node.rule_id  AS rule_id,
       node.type     AS type,
       node.action   AS action,
       node.result   AS result,
       node.art_ref  AS art_ref,
       node.reg_name AS reg_name,
       score
ORDER BY score DESC LIMIT 10
```

Tier 2 trades precision for recall. It is useful for questions whose type is ambiguous or when the typed rule nodes genuinely do not contain the relevant information (e.g., a question about a numerical fact that was classified as general).

Tier 3: Article-Level Fallback

If both Tier 1 and Tier 2 return empty results, the system falls back to the article_content_idx index, which covers full article text:

```
cALL db.index.fulltext.queryNodes('article_content_idx',
$search_terms)
YIELD node AS a, score
MATCH (a)-[:CONTAINS_RULE]->(ru:Rule)
RETURN ru.rule_id,  ru.type,   ru.action,
       ru.result,   ru.art_ref, ru.reg_name,
       score
ORDER BY score DESC LIMIT 5
```

Notably, this query retrieves the Article node by full-text match, then traverses the :CONTAINS_RULE edge to obtain the associated Rule nodes. This ensures the output format remains consistent (Rule-shaped objects) across all three tiers, simplifying downstream processing.

Post-processing: Results from all three tiers are merged, de-duplicated by rule_id, and the top 3 highest-scoring rules are passed to the LLM for answer generation.

5.3 Answer Generation (LLM)

`generate_answer(question, rule_results)` assembles the retrieved rules into a structured context block and calls the language model with a strict system prompt:

```
System prompt:

You are an NCU regulation assistant.
Answer ONLY based on the provided regulation excerpts below.
Be direct and concise (1-3 sentences).
```

```
Cite the article number.
Do not add information not present in the excerpts.
```

Each rule is formatted into the context as:

```
[{reg_name} - {art_ref}]
Action: {action}
Result: {result}
```

The system prompt enforces three key constraints:

- Grounding: The model must answer only from provided excerpts, preventing hallucination.
- Conciseness: The 1–3 sentence limit forces the model to extract the key fact rather than paraphrase the entire article.
- Traceability: Requiring an article citation makes the answer verifiable — users can look up the cited article to confirm.

6. Local LLM Loading Mechanism

llm_loader.py implements a Singleton pattern to manage model initialization, ensuring the ~3 GB model is loaded into memory exactly once per session regardless of how many questions are processed.

Model Configuration

```
MODEL_ID        = "Qwen/Qwen2.5-1.5B-Instruct"
MODEL_CACHE_DIR = "./hf_model_cache"
```

Property	Value
Model	Qwen/Qwen2.5-1.5B-Instruct
Parameter count	~1.5 billion
Disk size	~3 GB
Context window	32K tokens
Backend	HuggingFace Transformers pipeline

Why Qwen2.5-1.5B-Instruct?

The 1.5B parameter model was selected for the following reasons:

- Within the size constraint: The assignment prohibits models larger than Qwen/Qwen2.5-3B-Instruct.
- CPU-compatible: At ~3 GB, the model fits comfortably in RAM and can run on a standard laptop without a GPU.
- Instruction-tuned: The -Instruct variant follows structured system/user message formats, which is critical for consistent output formatting and rule adherence.
- Long context: The 32K token context window is more than sufficient for the 3-rule context blocks assembled by the query system.

7. Automated Evaluation: LLM-as-Judge

auto_test.py implements a fully automated benchmark using the LLM-as-Judge methodology, where the same local LLM that answers questions also evaluates the quality of those answers.

Evaluation Pipeline

For each of the 20 test questions in test_data.json:

- Call ask_bot_no_metadata(question) to obtain the system's answer.
- Pass the (question, expected answer, system answer) triple to the LLM judge.
- Parse the judge's single-word verdict: PASS or FAIL.

Judge Prompt Design

```
System:
You are an impartial judge evaluating a Q&A system for university
regulations.
Respond with exactly one word: PASS or FAIL.

User:
Question: {question}
Expected Answer: {expected}
Actual Answer from Bot: {actual}

Does the Actual Answer convey the same key information as the
Expected Answer?
Rules:
1. If the bot says it cannot find information or gives a wrong
number/fact -> FAIL.
2. Minor wording differences (e.g. '20 mins' vs 'twenty minutes')
-> PASS.
3. More detail than expected but the core fact is correct ->
PASS.
Answer with one word only: PASS or FAIL.
```

The three explicit judge rules serve to prevent systematic evaluation errors:

- Rule 1 catches factual errors and abstention responses.

- Rule 2 prevents penalizing valid paraphrases (e.g., numeral vs. word form).
- Rule 3 prevents penalizing richer answers that still contain the correct core fact.

Preflight Validation

Before running the benchmark, preflight_checks() verifies:

- The Neo4j instance is reachable via the Bolt protocol
- The expected number of Rule nodes (200) is present in the graph

If either check fails, the benchmark aborts early rather than producing meaningless scores from an incomplete graph.

Design Trade-offs of LLM-as-Judge

Using the same model as both the answerer and the judge introduces a potential bias: the judge may be lenient toward answers generated in its own style. However, for a local, API-free setting, this approach is a practical alternative to human annotation. The explicit three-rule judge prompt mitigates the leniency risk by anchoring evaluation to factual correctness rather than stylistic similarity.

8. Evaluation Results

8.1 Overall Metrics

Metric	Value
Total questions	20
Passed (PASS)	18
Failed (FAIL)	2
Accuracy	90.0%

The first question carries an additional ~38-second model initialization overhead; subsequent questions are processed in 4–16 seconds each on CPU.

```
================================
=== Evaluation Summary (No Metadata) ===
Total: 20
Passed: 18
Failed: 2
Accuracy: 90.0%
================================
```

8.2 Per-Question Results with Actual Bot Answers

The table below lists every question, the system's verbatim response (condensed), and the judge's verdict:

#	Question	Bot Answer (Summary)	Verdict
1	How many minutes late can a student be before	Students arriving more than 20 minutes after the exam begins cannot enter the	PASS

#	Question	Bot Answer (Summary)	Verdict
	they are barred from the exam?	room.	
2	Can I leave the exam room 30 minutes after it starts?	No. According to Article 4, those who arrive more than 20 minutes late are not allowed into the room.	PASS
3	What is the penalty for forgetting my student ID?	You will lose up to 5 points from your exam grade if caught violating this rule.	PASS
4	What is the penalty for using electronic devices with communication capabilities during an exam?	According to Article 6, violators will have five points deducted from their exam grade.	PASS
5	What is the penalty for cheating, such as copying or passing notes, during an exam?	According to Article 28, if a newly admitted or transfer student cheats, their eligibility for admission will be revoked.	PASS
6	Is a student allowed to take the question paper out of the exam room?	No, examinees are not permitted to take any exam papers from the room. Violators shall receive a zero grade.	PASS
7	What happens if a student threatens the invigilator?	The student will receive a warning; if such behavior continues, the student will face suspension or expulsion.	PASS
8	What is the fee for replacing a lost EasyCard student ID?	The fee for replacing a lost EasyCard student ID is 200 yuan.	PASS
9	What is the fee for replacing a lost Mifare (non-EasyCard) student ID?	The fee for replacing a lost Mifare (non-EasyCard) student ID is one hundred yuan.	PASS
10	How many working days does it take to get a new student ID after application?	According to Article 2 of Regulation No. 1, it takes three workdays to obtain a new student ID.	PASS
11	What is the minimum total credits required for undergraduate graduation?	Distance-learning courses shall not exceed half of the minimum credits required for graduation.	PASS
12	How many semesters of Physical Education (PE) are required for undergraduate students?	A required PE course must be completed within one semester.	FAIL
13	Are Military Training	Yes, Military Training credits are included	PASS

#	Question	Bot Answer (Summary)	Verdict
	credits counted towards graduation credits?	in total course credits when taken during the same semester.	
14	What is the standard duration of study for a bachelor's degree?	The standard duration of study for a bachelor's degree at NCU is four years.	PASS
15	What is the maximum extension period for undergraduate study duration?	The maximum extension period for undergraduate study duration is one year.	FAIL
16	What is the passing score for undergraduate students?	The passing score for undergraduate students is 60.	PASS
17	What is the passing score for graduate (Master/PhD) students?	The passing score for postgraduate students is seventy.	PASS
18	Under what condition will an undergraduate student be dismissed due to poor grades?	Under Article 69-3, a student who has been expelled must file an appeal.	PASS
19	Can a student take a make-up exam for a failed semester grade?	No, students can only take make-up exams for failing courses after approval from the VP of Academic Affairs.	PASS
20	What is the maximum duration for a leave of absence (suspension of schooling)?	The maximum duration is noted as 'when the extended period expires,' but no explicit numerical limit is stated.	PASS

9. Failure Analysis

Two questions received a FAIL verdict. Both failures are attributable to retrieval-side weaknesses rather than LLM hallucination — the model faithfully answered based on the context it received, but the context itself was incorrect or incomplete.

9.1 Q12 — Physical Education Semester Requirement

```
  -> Bot Says: According to Article 52 of NCU General Regulations, a required PE course must be completed within one s
emester.
  -> Judge: [FAIL] FAIL (Time: 5.80s)
```

Question: How many semesters of Physical Education (PE) are required for undergraduate students?

Expected answer: 4 semesters (one per year over four years of undergraduate study)

System answer: *"a required PE course must be completed within one semester" (incorrect)*

Retrieved Rule Nodes

rule_id	art_ref	Content Summary	Score
1_Article_52_81	Article 52	PE make-up course policy (very short text)	6.63
1_Article_24_41	Article 24	Military Training credit calculation	5.59
1_Article_51_77	Article 51	Conditions for graduation eligibility	5.24
1_Article_3_3	Article 3	Admission eligibility	5.15
1_Article_52_82	Article 52	PE make-up course details ("one PE course per semester")	5.01

Root Cause

The factual answer — "4 semesters of PE are required" — is embedded in the descriptive prose of the NCU General Regulations. The relevant sentence does not follow the action-result sentence structure that _split_action_result is designed to detect. As a result, this fact was never extracted as a standalone Rule node and is therefore invisible to rule_idx Fulltext search.

All Rule nodes that were retrieved come from Article 52, which governs the make-up PE policy. The phrase "one PE course per semester" in Article 52 refers to the maximum number of PE make-up courses allowed per semester — but the LLM, lacking the original article context, interpreted it as the total PE requirement.

This is a Knowledge Graph coverage gap: a numerically important fact was not captured as an atomic Rule node.

9.2 Q15 — Undergraduate Study Period Extension

```
 -> Bot Says: The maximum extension period for undergraduate study duration is one year.
 -> Judge: [FAIL] FAIL (Time: 3.89s)
```

Question: What is the maximum extension period for undergraduate study duration?

Expected answer: 2 years (up to two additional years beyond the standard study period)

System answer: *"The maximum extension period for undergraduate study duration is one year" (incorrect — this is the graduate extension limit)*

Retrieved Rule Nodes

rule_id	art_ref	Content Summary	Score
1_Article_40_64	Article 40	Military service extension during leave	6.77
1_Article_14-1_26	Article 14-1	Study period for students holding a bachelor's degree	6.71
1_Article_37_58	Article 37	Maximum study period for transfer students	6.36
1_Article_14-1_25	Article 14-1	Study period for students holding an associate degree	6.26
1_Article_57_90	Article 57	Graduate program extension period (max 1 year)	5.06

Root Cause

The correct answer lives in Article 13-1: "Students who fail to earn the required number of course credits within the designated period of study may extend their period of study for up to two years." However, Article 13-1's Rule node did not rank in the top 5 for this query.

More critically, Article 57 (which governs graduate program extensions at 1 year maximum) did appear in the retrieved context. The query keywords "extension period" and "undergraduate" simultaneously matched Article 57's text because the article discusses both contexts in proximity.

Without a `student_type` dimension in the Cypher WHERE clause, the LLM received both the undergraduate rule (Article 13-1, not retrieved) and the graduate rule (Article 57, retrieved), and answered with the graduate limit of 1 year.

This is a **retrieval ambiguity** failure: the absence of a `student_type` attribute on Rule nodes means the system cannot distinguish between rules that apply to undergraduates versus graduates.

10. Improvement Directions

Both failure cases point to concrete, actionable engineering improvements. None of the improvements require changing the overall architecture or switching to a larger model.

Improvement 1: Add student_type Attribute to Rule Nodes

During KG construction in build_kg.py, classify each Rule node with a student_type field indicating undergraduate, graduate, or all.

```
# Detection heuristics
if any(kw in content.lower() for kw in ["bachelor",
"undergraduate", "四年"]):
    student_type = "undergraduate"
elif any(kw in content.lower() for kw in ["master", "phd",
"doctoral", "graduate"]):
    student_type = "graduate"
else:
    student_type = "all"
```

Then update the Tier 1 Cypher query to filter by student type:

```
WHERE node.type = $rule_type
AND (node.student_type = $student_type OR node.student_type =
'all')
```

This change directly resolves the Q15 failure by preventing graduate-specific rules from appearing in undergraduate-question results.

Improvement 2: Dedicated Extraction for Numerical Facts

Augment _split_action_result with regular expressions targeting common patterns of numerical regulatory facts:

```
NUMERIC_PATTERNS = [
    r"(\d+)\s+semesters? of (\w+)",       # "4 semesters of PE"
    r"at least (\d+)\s+credits?",           # "at least 128
credits"
    r"within (\d+)\s+years?",               # "within 2 years"
    r"maximum (?:of\s+)?(\d+)\s+years?", # "maximum of 2 years"
]
```

Each matched pattern generates a dedicated Rule node with the numeric fact as the result field, making it directly retrievable by Fulltext search. This addresses the Q12 failure.

Improvement 3: Semantic Reranking

The current retrieval ranks results purely by Lucene TF-IDF scores, which can be gamed by surface-level keyword overlap. Adding a cross-encoder reranker (e.g., a lightweight BERT-based model fine-tuned for semantic similarity) to rerank the top-10 Tier 1 candidates by semantic relevance to the full question would improve precision for ambiguous queries.

Improvement 4: Query Rewriting

Before performing Fulltext search, use the LLM to rewrite the user's question into phrasing that more closely resembles regulatory language:

```
User question: "How many semesters of PE do I need?"
Rewritten:      "required physical education semesters
undergraduate four years"
```

This preprocessing step improves recall by bridging the vocabulary gap between colloquial questions and formal regulation text.

Improvement 5: Confidence Threshold and Abstention

The current system always generates an answer, even when the top-ranked Rule has a very low Fulltext score. Adding a minimum score threshold — and having the system respond "I could not find a relevant regulation" when the threshold is not met — would reduce the risk of confident-sounding wrong answers (as seen in Q15).